

ONE SET OF WEIGHTS, MANY ALGORITHMS: HOW LOOPED TRANSFORMERS ROUTE COMPUTATION ACROSS LOOPS

Anonymous authors

Paper under double-blind review

ABSTRACT

Looped Transformers repeatedly apply the same set of Transformer layers, providing a recurrent architecture for latent computation. Their strong performance on iterative reasoning and length-generalization tasks suggests an appealing explanation. Recurrence may provide an inductive bias: the model reuses a learned algorithm across loops. However, weight sharing alone does not imply that every loop executes the same operation. This raises a basic question: what computation is actually reused across loops, and what determines how the shared parameters are used at each loop? We study these questions in Looped Transformers. In a graph-walk setting, we first show that successive loops need not perform the same operation: under the same shared parameters, different loops can advance the computation by different amounts or remain at an already reached target. Thus, the same shared parameters can encode different algorithms across loops. We then show that these behaviors can be controlled through the hidden state entering each loop. A learned linear layer applied to this state can redirect the next frozen loop toward different targets, showing that the loop input helps select how the shared parameters are used. Going further, attention interventions reveal a mechanism for this selection in the selected models: patching attention patterns from a steered run into an unsteered run recovers much of the steering effect, providing causal evidence that attention routing carries the change in algorithmic behavior. Finally, we examine depth generalization under steering and test whether steering depends on computation learned by the backbone under different supervision schemes.

1 INTRODUCTION

Recent work on *latent reasoning* studies how models can carry out multi-step reasoning within their hidden states, without generating every intermediate step as text (Saunshi et al., 2025; Geiping et al., 2025). Recurrent architectures provide a natural way to support this kind of computation: the same parameters can be applied repeatedly while the internal state continues to evolve. Looped Transformers bring this idea to Transformer models by reusing the same set of Transformer layers over multiple loops (Dehghani et al., 2019; Yang et al., 2024). This recurrent design provides a useful inductive bias for iterative computation. Looped Transformers can learn iterative learning algorithms and multi-step optimization procedures (Yang et al., 2024; Gatmiry et al., 2024), improve length generalization on tasks with iterative solutions (Fan et al., 2025), and use additional loops as a source of latent test-time computation at language-model scale (Geiping et al., 2025; Zhu et al., 2025). These results suggest that recurrent depth can provide a way to reuse learned computation over multiple internal steps.

These results naturally suggest an iterative view of Looped Transformers: the shared layers may repeatedly apply the same learned procedure as computation progresses. For example, in a graph walk, one might expect every loop to move one step along the path. Yet weight sharing does not imply such a fixed correspondence. The parameters are shared across loops, but the hidden state entering those parameters changes as computation proceeds. The same network is therefore applied to different internal states, so weight sharing alone does not specify what operation each loop performs. The training objective can also leave the intermediate trajectory underdetermined. When only the final answer is supervised, the loss does not require the model to follow any particular sequence of

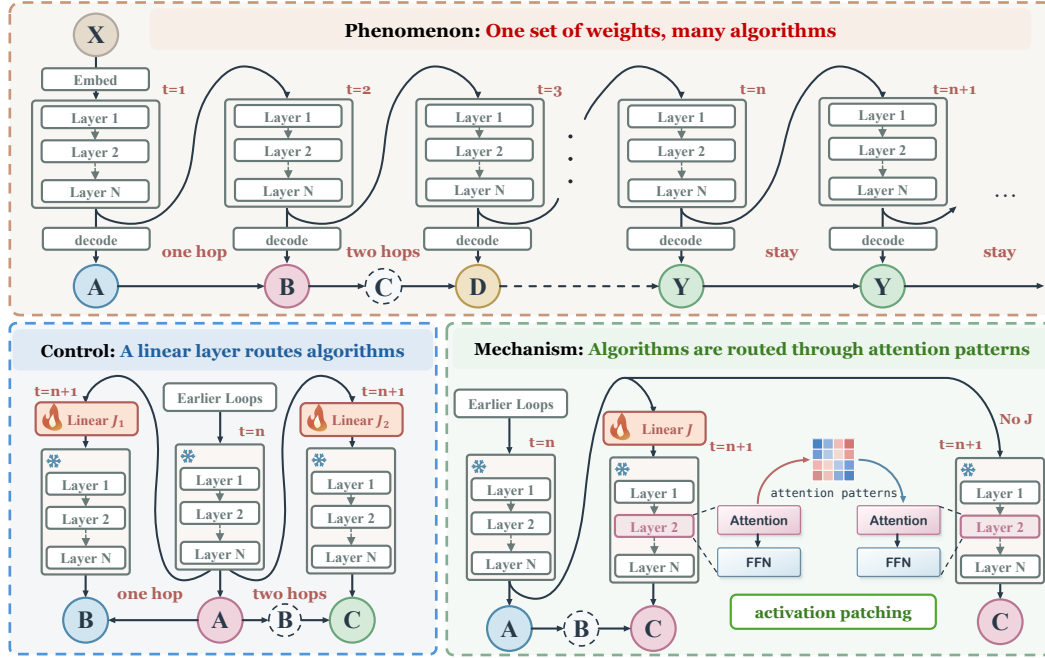


Figure 1: **One set of weights, different targets of computation.** Top: task progress varies across loops. Bottom left: learned linear layers select different targets for the same frozen loop. Bottom right: attention-pattern patching tests the role of routing in target selection. Flames mark trainable components; snowflakes mark frozen components.

intermediate steps. This leads to a basic question: *what computation is actually reused across loops, and what determines how the shared parameters are used at each loop?* Answering this question is important for understanding the inductive bias of recurrent depth and the internal process of latent reasoning. It also affects when additional loops provide useful computation and whether recurrent test-time computation can be controlled.

Prior work has gradually moved from asking whether recurrence can implement algorithms to studying how learned computation is organized across loops. Constructive results show that fixed-weight Looped Transformers can execute multi-step programs, while learning-based analyses show that looped models can learn iterative optimization procedures (Giannou et al., 2023; Yang et al., 2024; Gatmiry et al., 2024). These results establish that shared parameters can support reusable algorithmic computation, but they do not specify how that computation is distributed across loops after training. Mechanistic work therefore turns to the learned recurrent trajectory itself. Blayney et al. (2026) identify layer-wise fixed points, cyclic latent trajectories, and repeated stages of inference across loops. Probing studies likewise find that intermediate recurrent states do not always form a simple, directly readable latent chain of thought (Lu et al., 2025). More recently, Zhang et al. (2026) study task progress across loops directly. They show that training can select computation frontiers with different speeds, so the amount of progress associated with one loop is learned rather than fixed by weight sharing. Together, these studies characterize how recurrent computation is learned and organized across loops, but leave open whether the state entering a loop can causally select what the same frozen layers compute next, and how this selection is implemented inside the model.

We address these gaps by testing whether the entering hidden state can redirect the next computation of a fixed frozen loop, and by tracing this control to attention routing (Figure 1). We begin with the recurrent trajectories that arise without intervention. In graph-walk tasks, different loops can advance by different numbers of graph steps or remain at an already reached target. A learned trajectory therefore need not follow a fixed “one loop = one algorithmic step” rule (Section 3). This variation makes the entering hidden state a natural candidate for controlling the next computation. We freeze the shared Transformer layers and learn a linear layer at the loop boundary. Changing only the entering state can redirect the next frozen loop toward different targets (Section 4). Having established

this control, we then ask how it is implemented inside the loop. Patching attention patterns from a steered run into an unsteered run recovers much of the steering effect, providing causal evidence that attention routing carries an important part of the resulting behavior change (Section 5). Finally, we examine how far steering extends beyond its training depth and whether its success depends on computation already learned by the backbone. Comparing backbones trained under different supervision schemes tests our interpretation that J selects learned computation (Section 6).

2 RELATED WORK

Recurrent computation. Recurrent architectures increase computation depth by repeatedly applying the same parameters (Dehghani et al., 2019). Prior work shows that looped Transformers can execute programs and learn iterative algorithms (Giannou et al., 2023; Yang et al., 2024; Gatmiry et al., 2024), improve length generalization (Fan et al., 2025), and use additional loops to improve reasoning (Saunshi et al., 2025; Geiping et al., 2025; Zhu et al., 2025). These results establish the value of recurrent computation. They do not, however, determine what operation each loop performs in a trained model.

Computation learned across loops. Understanding loop behavior requires distinguishing the algorithms an architecture can express from the computation selected by training. Training budgets can select different rates of task progress within the same shared architecture (Zhang et al., 2026). Studies of looped language models also identify cyclic latent trajectories and repeated stages of inference (Blayney et al., 2026). Intermediate readouts need not directly reveal this internal computation: their interpretation can depend on the layer and decoding method (Lu et al., 2025). These findings motivate going beyond observing an existing trajectory. We ask whether changing the hidden state entering a loop can make the same parameters perform different task-level operations.

Supervision of recurrent states. Supervision provides a way to examine how learned computation affects state control. Training objectives shape both the recurrent trajectory and the information available at intermediate readouts (Fan et al., 2026; Popescu et al., 2026). Supervising outputs does not fully specify the internal computation; even per-loop supervision can leave state variables such as hidden-state scale uncontrolled (Sharma & Vu, 2026). We therefore compare backbones trained under different supervision schemes to test whether steering depends on computational capabilities already learned by the backbone.

State steering and causal interventions. State interventions and activation patching provide complementary tools for testing this account. Steering modifies hidden states to control a frozen model’s behavior (Turner et al., 2023), including through learned linear layers with a bias (Singh et al., 2024). Activation patching replaces internal activations to test which components contribute to a behavior (Wang et al., 2023). We combine these tools by using a linear layer to change the loop input and select the next target, then patching attention patterns to test whether routing carries this selection. This extends the study of what recurrence can compute to whether learned computation can be selected through the entering state and how that selection is implemented.

3 PHENOMENON: TASK PROGRESS VARIES ACROSS LOOPS

We first examine whether successive loops make the same amount of task progress. We use graph traversal to track the node predicted after each loop.

Each input specifies a directed graph G with one successor per node, a starting node s , and a path length k . Let $f_G(v)$ denote the successor of v . The target is $f_G^k(s)$. Appendix A.1 gives the input format and examples.

The models reuse a two-layer Transformer block. We denote each configuration by $D_m L_n$, where m is the maximum training path length and n is the number of training loops. D8L8 and D8L6 both train on paths of length 1–8. They use eight and six loops, respectively, with supervision only on the final answer.

To observe progress within and beyond the training loop budget, we request the eight-hop target and read out predictions at loops 0–16. Loop 0 denotes the state before the first block application. Each

model is evaluated on 512 held-out ten-node cycles with all ten starting nodes, giving 5,120 examples. Figure 2 shows four runs from twelve D8L8 and five D8L6 backbones. Rows indicate loops, columns indicate nodes along the path, and color shows prediction frequency.

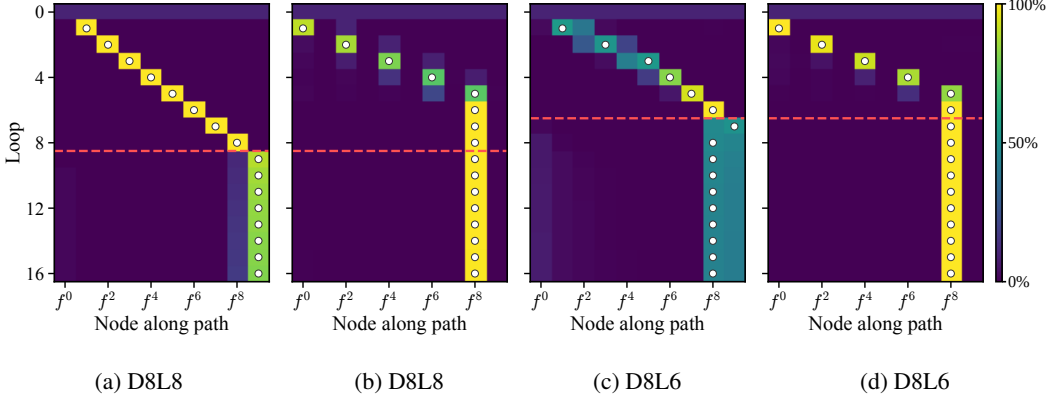


Figure 2: Intermediate readouts; color shows prediction frequency. Red dashed lines mark the last supervised loop; white dots mark the unique most frequent prediction.

The readout trajectories differ both before and after reaching the target. In Figure 2a, the most frequent prediction advances one node per loop. In Figure 2b, it initially stays at the start and then advances mainly two nodes at a time. Both D8L6 models reach the target in six loops but predict different intermediate nodes (Figures 2c, 2d). After the last supervised loop, some models keep predicting the target while others predict nodes beyond it.

Shared weights and final-answer supervision therefore do not impose a fixed relation between loop number and graph hops. These readouts identify predicted nodes, not the hidden operations that produce them. We next test whether changing the loop input can control the next predicted node. The target-selection and main graph mechanism experiments use the same D8L6 backbone.

4 CONTROL: STATE STEERING REDIRECTS THE NEXT TARGET OF A FROZEN LOOP

The different trajectories in Section 3 motivate a causal test. We ask whether changing the hidden state entering a loop can select its next target while keeping the network weights and loop budget fixed.

4.1 STATE STEERING WITH A FROZEN BACKBONE

Let F denote the shared Transformer block and h_t the sequence of token hidden states after loop t . The recurrence is

$$h_{t+1} = F(h_t), \quad t = 0, 1, 2, \dots \quad (1)$$

We apply the same learned linear layer J , including a bias, to each token’s hidden state before the next block. The map does not mix information across tokens. We train only J and keep the underlying network, which we call the backbone, frozen.

4.2 SELECTING ONE-HOP AND TWO-HOP TARGETS

D8L6 readouts can advance by one or two graph hops in a loop (Figures 2c, 2d). We test whether J can select between these continuations.

We train two linear layers on the same state h_6 . Each map has a diagonal term and a rank-48 low-rank term. Let $u = f_G^8(s)$ be the starting node for continuation. We train J_{one} and J_{two} so that the next frozen loop predicts $f_G(u)$ and $f_G^2(u)$, respectively. Appendix A.2 gives the configuration.

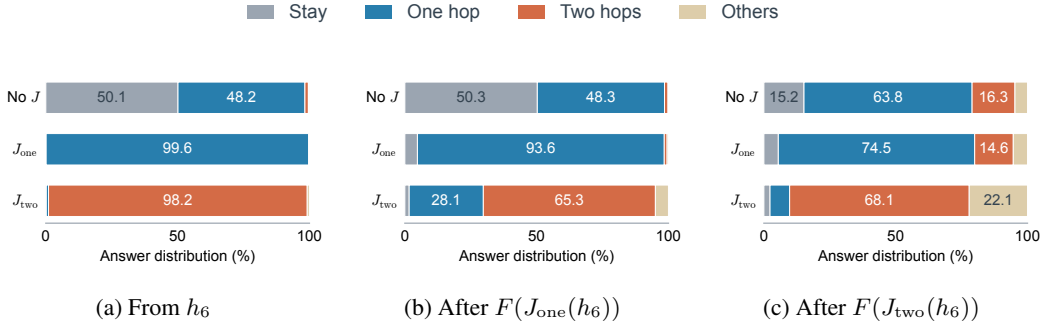


Figure 3: Answer distributions after one frozen loop from three starting states. Rows select the map applied before that loop. In (a), the reference node is u . In (b) and (c), it is the first map’s intended target, $f_G(u)$ and $f_G^2(u)$, respectively. Stay, one hop, and two hops are measured from these reference nodes. Others includes all remaining answers. No J in (b,c) omits only the second map. The steered results average two map fits. Panel (a) uses 4,110 examples; panels (b,c) use 3,200 examples with distinct nodes from u through $f_G^4(u)$. Appendix A.4.3 reports individual fits and confidence intervals.

The maps direct the next frozen loop toward their respective targets without changing the input graph (Figure 3a). This control also holds on graphs excluded from map training.

4.3 COMPOSING TARGET-SPECIFIC CONTROLLERS

We next reuse these maps to control a second loop, without further training:

$$h' = F(J_a(h_6)), \quad h'' = F(J_b(h')), \quad a, b \in \{\text{one}, \text{two}\}. \quad (2)$$

The second map acts on h' . We neither reset the state nor supply an intermediate answer. The one–one, one–two, two–one, and two–two sequences target nodes two, three, three, and four hops beyond u , respectively. Matched controls keep the first map and both loops but omit the second map.

The maps also control the second continuation (Figures 3b,3c). One–one reaches its target in 93.6% of examples, compared with 48.3% without the second map. One–two and two–one reach the same three-hop target at different rates: 65.3% and 74.5%. Thus, accuracy depends on the order of the maps. Figure 3 reports all four sequences and their controls.

The final target becomes readable after the second frozen loop. Immediately after the second map, before F , target accuracy is only 0.2–1.6%. Together, these results show that maps trained for one continuation can control a second without composition training. Appendix A.4.3 gives the evaluation protocol and results across all five backbones. Longer controller sequences remain untested.

Table 1 summarizes the further tests of steering and its limits, all with frozen backbones.

Table 1: Further experiments on state control.

Task	Form of J	Focus
Ouro letter-walk	Dense linear layer	Routing mediation of steering (§5)
Parity	Diagonal + low-rank ($r = 48$)	Readout timing at longer lengths (§6.1)
Graph continuation	Diagonal + low-rank ($r = 48$)	Control beyond the fitted loops (§6.1)
Ouro cancellation	Residual linear layer ($r = 128$)	Dependence on learned computation (§6.2)

5 MECHANISM: ATTENTION ROUTING CARRIES THE STEERING EFFECT

We have shown that the entering hidden state can control the next target. We now test whether attention routing carries this effect by changing where the network reads. We use attention interventions in graph traversal and Ouro letter-walk.

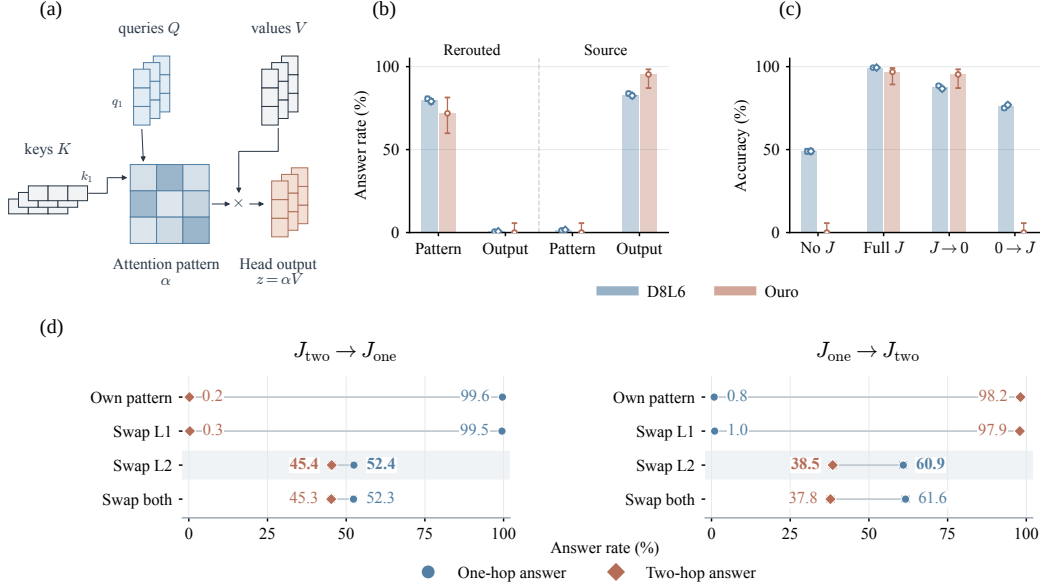


Figure 4: Attention interventions. (a) Attention weights α combine values V . (b) Cross-graph pattern and head-output patches. (c) Pattern exchange between steered (J) and unsteered (0) runs. (d) Pattern exchange between one-hop and two-hop runs. Arrows point from source to receiver. In (d), L1 and L2 denote the two layers of the final loop; intervals are graph-bootstrap 95% confidence intervals.

For graph traversal, we reuse the D8L6 backbone and maps from Section 4. We also test Ouro-2.6B on *letter-walk*. Each input specifies who passes a letter to whom and who initially holds it. The model must identify the holder after a given number of steps (Appendix B).

We fine-tune Ouro on 1–4-step questions using four loops and final-only supervision. We then freeze the backbone and train a dense linear layer J on 1–8-step questions with the same loop budget and final readout. The same map acts on each token before loops 2–4. Appendices A.3, B.2, and B.3 give the training and intervention protocols. Figure 4b,c shows the two D8L6 map fits separately and one Ouro estimate.

5.1 PATTERN AND OUTPUT PATCHES SELECT DIFFERENT ANSWERS

We first separate where an attention head reads from what it retrieves. Let α denote its attention pattern, V its value vectors, and $z = \alpha V$ its output (Figure 4a). A pattern patch copies the source’s attention weights but keeps the receiver’s values. An output patch instead copies the source’s head output:

$$z_{\text{pattern patch}} = \alpha_{\text{source}} V_{\text{receiver}}, \quad z_{\text{output patch}} = z_{\text{source}}. \quad (3)$$

The source and receiver have aligned token positions but different graphs and starting nodes. Suppose a position selected by the source pattern contains D in the source and C in the receiver. A pattern patch should favor C, whereas an output patch should favor D. We require C, D, and the original receiving answer to be distinct so that these outcomes can be distinguished. Ouro uses the same construction with letter-walk descriptions. In both tasks, the receiver continues from the patched activation.

The two patches produce the predicted answers (Figure 4b). Pattern patches produce the rerouted answer, C in the example above, in 79.8% of D8L6 cases and 71.9% of Ouro cases. Output patches produce the source answer, D, in 83.2% and 95.3% of cases, respectively. Changing where a head reads can therefore redirect the answer without copying the source’s retrieved content.

5.2 ATTENTION PATTERNS CARRY THE STEERING EFFECT

We next test whether routing carries the effect of J . The source and receiver now share the same input. One run is steered and the other is not.

We copy attention patterns between selected heads in both directions. Each receiver keeps its own values and steering schedule:

$$\begin{aligned} z_J &= \alpha_J V_J, & z_0 &= \alpha_0 V_0, \\ z_{J \rightarrow 0} &= \alpha_J V_0, & z_{0 \rightarrow J} &= \alpha_0 V_J. \end{aligned} \quad (4)$$

Here J and 0 denote steered and unsteered runs. The arrow points from source to receiver. If routing carries the effect, $J \rightarrow 0$ should recover steering gains, whereas $0 \rightarrow J$ should reduce steered accuracy.

Both effects occur in the selected models (Figure 4c). In the seed-6 D8L6 backbone, copying steered patterns raises unsteered accuracy from 49.0% to 87.6%, compared with 99.4% under full steering. This recovers 76.5% of the gain, averaged across two fits. The reverse patch reduces steered accuracy to 76.0%. In Ouro, the forward patch raises accuracy from 0/64 to 61/64, close to 62/64 under full steering. The reverse patch reduces accuracy to 0/64. Thus, copying steered patterns recovers much of the steering effect without applying J in the receiving run.

The routing effect depends on the backbone. It is strongest in seed 6, whose native readouts advance by different amounts across loops. Seeds 3, 5, and 7 instead show a regular two-hop readout pattern before retaining the requested endpoint. In these backbones, one-hop steering succeeds, but transferring its attention patterns to an unsteered run fails, even with dense affine maps and every subset of heads in the final loop. Thus, successful target control does not imply that patterns alone can transfer it. Appendix A.4.4 relates these failures to the native readout trajectories.

5.3 ATTENTION ROUTING SWITCHES THE NEXT TARGET

Finally, we test whether routing carries the choice between one-hop and two-hop targets. From the same h_6 , we run $F(J_{\text{one}}(h_6))$ and $F(J_{\text{two}}(h_6))$. We exchange attention patterns within the final frozen loop while keeping the receiver’s values:

$$\begin{aligned} z_{\text{one}} &= \alpha_{\text{one}} V_{\text{one}}, & z_{\text{two}} &= \alpha_{\text{two}} V_{\text{two}}, \\ z_{\text{two} \rightarrow \text{one}} &= \alpha_{\text{two}} V_{\text{one}}, & z_{\text{one} \rightarrow \text{two}} &= \alpha_{\text{one}} V_{\text{two}}. \end{aligned} \quad (5)$$

Here one and two denote runs steered by J_{one} and J_{two} , respectively. The arrow points from source to receiver. The graph, backbone weights, and one-loop continuation budget remain fixed.

Second-layer patterns redirect the target in both directions (Figure 4d). Copying two-hop patterns into the one-hop run raises the frequency of two-hop answers from 0.2% to 45.4%. The reverse patch raises the frequency of one-hop answers from 0.8% to 60.9%. First-layer exchange has little effect. Exchanging both layers gives similar results to exchanging the second layer alone. These results average two fits on a separate set of 4,116 examples; the remaining responses are other answers. Appendix A.4.2 gives individual fits and broader comparisons.

Together, these interventions show that attention routing carries both steering gains and part of the change in target. Pattern patching does not fully reproduce the source run’s behavior. This suggests that the receiving state and the remaining computation also contribute. Head-output restoration provides a complementary check (Appendix A.4.1).

6 SCOPE AND MECHANISTIC TESTS OF STATE STEERING

We now examine the scope of state steering and test its dependence on learned computation. We first evaluate execution beyond the depths used to train J . We then compare backbones trained under different supervision schemes to test whether their learned computational behavior predicts steering success.

6.1 DEPTH GENERALIZATION ACROSS TASKS

We compare depth generalization in graph traversal and parity. Parity asks whether a binary sequence contains an even or odd number of ones: $y = (\sum_{i=1}^n x_i) \bmod 2$. Following Fan et al. (2025), we read out the answer after $t = n$ loops. We train the backbone on lengths 1–20, then freeze it and train a shared linear layer J on lengths 20–40 (Appendix C).

The two tasks differ beyond the range used to train J (Figure 5). In the displayed parity run, accuracy remains high at lengths approaching 200. In graph traversal, J is trained to maintain one-hop progress through loop 16. Accuracy drops substantially before loop 20 despite continued steering. This decline appears across twelve D8L8 backbones, separate from the D8L6 models used in the mechanism experiments (Appendix D).

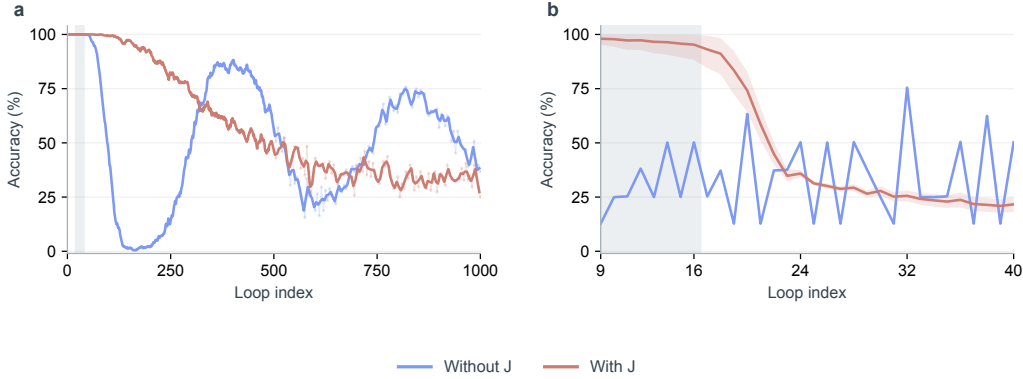


Figure 5: Accuracy with and without steering on (a) parity and (b) graph continuation. Blue denotes no J ; red denotes steering with J .

To understand the parity result, we examine when accurate readouts occur relative to the requested schedule $t = n$. Without steering, the accuracy band shifts earlier relative to this schedule as length increases. In seed 2, it lies roughly half a loop early near length 60. Steering brings the band closer to the diagonal and shifts the best nearby integer readout from $t = n - 1$ to $t = n$ (Figure 6a,b).

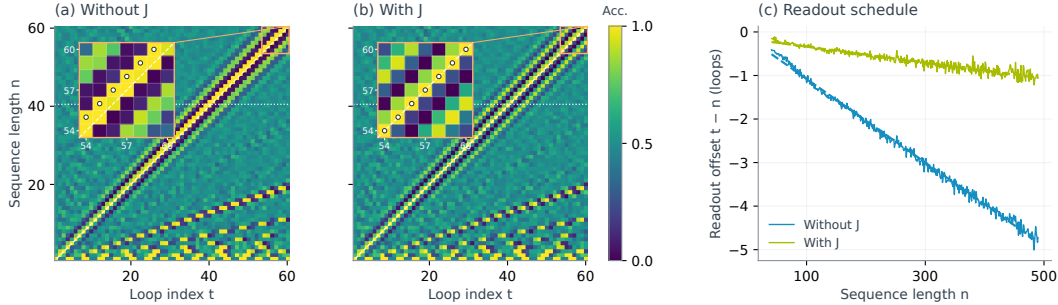


Figure 6: Parity readout timing for seed 2. (a,b) Accuracy across input length n and loop t , without and with J ; insets enlarge lengths 54–60. The diagonal marks $t = n$, and dots mark the best nearby integer loop. (c) Continuous readout offset $t - n$ and fitted trends across lengths 41–490.

We quantify this shift as $\Delta(n) = t_{\text{read}}(n) - n$, where t_{read} estimates the position of the recurring accuracy band. Steering raises the fitted slope of $t_{\text{read}}(n)$ against n from 0.9904 to 0.9982. The magnitude of the timing drift therefore falls from 0.96 to 0.18 loops per 100 tokens (Figure 6c). This supports a timing explanation: J reduces an error that accumulates with length. Appendices C.3 and C.4 describe the estimator and report results for other backbones.

6.2 DOES STEERING DEPEND ON LEARNED COMPUTATION?

We test whether J selects computation already learned by the backbone. An alternative is that J and F jointly implement a new computation. Under our interpretation, steering should depend on the computational behavior learned before J is introduced. We therefore compare backbones with the same architecture but different training supervision.

We use Ouro antonym cancellation. Each step removes the leftmost adjacent antonym pair and joins the remaining words in order, which can create another adjacent pair. The model must name the pair removed at the requested step. Both backbones train for four loops on deletion steps 1–4. Stepwise supervision trains loop t to report deletion t . Final-only supervision trains the backbone to answer a requested deletion step only at loop 4. Appendix E gives the training and evaluation protocols.

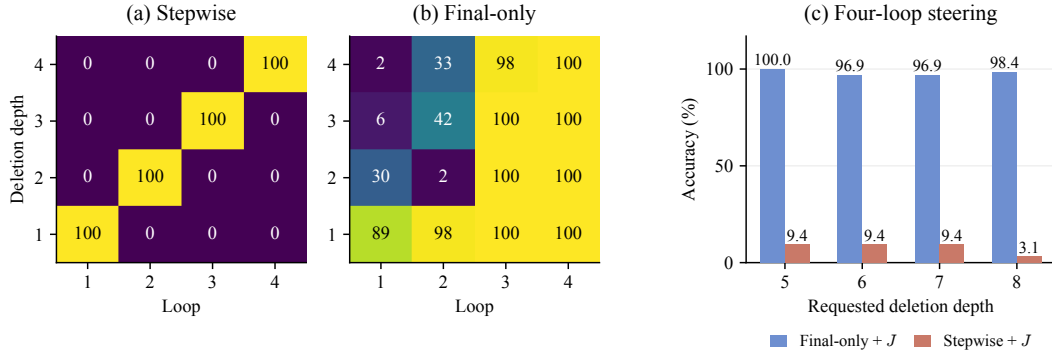


Figure 7: (a,b) Native Ouro readout accuracy under stepwise and final-only supervision. (c) Steering deeper deletions with four loops. Values are accuracies (%).

The native readouts show different computational behavior before steering (Figure 7a,b). The stepwise backbone predicts the deletion assigned to each loop, with no accurate readout of later deletions. The final-only backbone already answers 255/256 requests at loop 3, including 63/64 for deletion 4. It can therefore answer in fewer loops than the one-deletion-per-loop schedule. If J selects this learned capability, we expect it to support deeper answers within four loops in the final-only backbone, with much less success in the stepwise backbone.

We freeze both backbones and train a separate shared rank-128 residual linear layer for each. Each map receives 500 updates on deletion steps 1–8, with exactly four loops per request. Steps 5–8 are beyond backbone training but are included in J training.

The steering results match this prediction (Figure 7c). On the same held-out sequences at steps 5–8, the final-only backbone answers 251/256 requests correctly with J (98.0%), compared with 20/256 (7.8%) for the stepwise backbone. At step 8 alone, the counts are 63/64 and 2/64, respectively, within the same four-loop budget. Both backbones score zero without J on steps 5–8.

Thus, steering success depends on the trained backbone even when its architecture, controller family, and controller training budget are held fixed. The agreement between native readouts and steering outcomes supports the view that J selects learned computation. It does not establish that stepwise training excludes all multi-step computation or rule out a new computation formed jointly by J and F .

AI USE STATEMENT

We used ChatGPT and Codex to assist with code implementation, literature search, and manuscript drafting and editing. The authors take responsibility for the final content and reported results.

ETHICS STATEMENT

The experiments study recurrent computation using synthetic graph, parity, and rule-based language tasks. They do not involve human participants. The attention interventions are used to analyze model computation; the experiments do not evaluate the safety of deploying steered models in downstream applications.

REPRODUCIBILITY STATEMENT

The appendices document task definitions, input and output formats, supervision, steering configurations, and evaluation protocols for graph traversal (Appendix A), Ouro letter-walk (Appendix B), parity (Appendix C), graph continuation (Appendix D), and Ouro antonym cancellation (Appendix E). The corresponding appendices report sample counts, selection rules, uncertainty estimates, and additional runs where available, and distinguish evaluations beyond backbone training from those beyond steering-map training.

REFERENCES

- Hugh Blayney, Álvaro Arroyo, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, Michael M. Bronstein, and Xiaowen Dong. A mechanistic analysis of looped reasoning language models. *arXiv preprint arXiv:2604.11791*, 2026. URL <https://arxiv.org/abs/2604.11791>.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for length generalization. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=2edigk8yoU>.
- Ying Fan, Anej Svete, and Kangwook Lee. Bridging the gap between latent and explicit reasoning with looped Transformers. *arXiv preprint arXiv:2606.31779*, 2026. URL <https://arxiv.org/abs/2606.31779>.
- Khashayar Gatmiry, Nikunj Saunshi, Sashank J. Reddi, Stefanie Jegelka, and Sanjiv Kumar. Can looped Transformers learn to implement multi-step gradient descent for in-context learning? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 15130–15152. PMLR, 2024. URL <https://proceedings.mlr.press/v235/gatmiry24b.html>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3b01972cf31e6fa0fe29e4b8b5c2a0a1-Abstract-Conference.html.
- Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped Transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 11398–11442. PMLR, 2023. URL <https://proceedings.mlr.press/v202/giannou23a.html>.

- Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. Latent chain-of-thought? decoding the depth-recurrent transformer. *arXiv preprint arXiv:2507.02199*, 2025. URL <https://arxiv.org/abs/2507.02199>.
- Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive depth in looped Transformers: Diagnosing learned halting gates and trajectory readouts. *arXiv preprint arXiv:2607.20519*, 2026. URL <https://arxiv.org/abs/2607.20519>.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped Transformers. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2502.17416>.
- Rituraj Sharma and Tu Vu. Dense supervision is not enough: The readout blind spot in looped language models. *arXiv preprint arXiv:2606.24898*, 2026. URL <https://arxiv.org/abs/2606.24898>.
- Shashwat Singh, Shauli Ravfogel, Jonathan Herzig, Roei Aharoni, Ryan Cotterell, and Ponnurangam Kumaraguru. Representation surgery. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 45663–45680, 2024. URL <https://proceedings.mlr.press/v235/singh24d.html>.
- Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023. URL <https://arxiv.org/abs/2308.10248>.
- Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.
- Liu Yang, Kangwook Lee, Robert D. Nowak, and Dimitris Papailiopoulos. Looped Transformers are better at learning learning algorithms. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HHbRxoDTxE>.
- Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When does recurrence become an algorithm? convergence selection in weight-tied looped Transformers. *arXiv preprint arXiv:2607.20594*, 2026. URL <https://arxiv.org/abs/2607.20594>.
- Rui-Jie Zhu, Zixuan Wang, Kai Hua, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025. URL <https://arxiv.org/abs/2510.25741>.

A GRAPH TRAVERSAL

This appendix supports Sections 3–5. It defines the graph task, specifies the target-selection and attention experiments, and reports additional results.

A.1 TASK DEFINITION AND INPUT FORMAT

The task predicts the node reached after a requested number of edges from a specified start. Each node has a discrete token ID. The input consists of a beginning token, one three-token record per directed edge, and a query:

```
BOS [EDGE source destination] ... QUERY start DEPTH[k]
ANSWER
```

The brackets group records for display; they are not input tokens. BOS, EDGE, QUERY, and ANSWER are dedicated tokens, as is each supported depth. The model predicts a node from the hidden state at ANSWER. A ten-node input has 35 tokens. The graph is supplied in the input, rather than stored as a fixed set of edges in the model.

For a concrete example, consider the ten-node cycle $0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$. Its edge records are EDGE 0 1, EDGE 1 2, and so on through EDGE 9 0. The query QUERY 0 DEPTH[8]

ANSWER has target node 8. Changing the start to 3 gives target node 1. These examples illustrate the encoding and are not reported evaluation samples.

Backbone training samples requested path lengths 1–8 and applies cross-entropy to the requested node at the final training loop: loop 8 for D8L8 and loop 6 for D8L6. Earlier readouts receive no task loss. For target selection, the request remains eight steps long. Starting from the same D8L6 state after loop 6, the one-hop map is supervised on the successor of the eight-step answer, and the two-hop map on its second successor, after one additional frozen loop. In the cycle example starting at 0, these labels are 9 and 0, respectively. No answer node is appended to the input.

The attention interventions use this same input format and target rule. They replace internal attention quantities in already trained runs; they do not introduce a new training target. Their paired inputs and scoring rules are specified in Appendix A.3.

A.2 MODEL AND STEERING CONFIGURATION

Backbone and intervention. The target-selection experiment uses the ten-node D8L6 backbone trained with seed 6 and saved at update 16,000. It receives a request for the eight-hop endpoint $u = f_G^8(s)$ and runs for six loops to produce h_6 . From this same state, we evaluate an unsteered seventh loop, $F(h_6)$, and the two steered alternatives, $F(J_{\text{one}}(h_6))$ and $F(J_{\text{two}}(h_6))$. The input, backbone weights, and one-loop continuation budget are fixed across conditions. Each map acts on every token state before this final frozen loop.

Map dimensions and training. Both maps use $J(h) = h(D + AB) + b$ with hidden dimension 256 and rank 48. The learned diagonal D has 256 parameters, $A \in \mathbb{R}^{256 \times 48}$, $B \in \mathbb{R}^{48 \times 256}$, and $b \in \mathbb{R}^{256}$, giving 25,088 trainable parameters per map. We fit each target map twice, with seeds 1 and 2. Training updates only the map, using cross-entropy on the target after the frozen loop: $f_G(u)$ for J_{one} and $f_G^2(u)$ for J_{two} . Each fit runs for 8,000 AdamW updates with batch size 128, learning rate 10^{-4} , zero weight decay, and gradient clipping at 1.0. Validation is performed every 400 updates, and the earliest checkpoint attaining the highest validation accuracy is retained. Training excludes the locked selection and confirmation graphs.

A.3 EVALUATION AND ATTENTION INTERVENTIONS

Native intermediate readouts. For Figure 2, we request the eight-step answer and evaluate the frozen models at loops 0–16. Loop 0 precedes the first shared block. Each model is evaluated on 512 held-out ten-node cycles with all ten starting nodes, giving 5,120 examples. The figure shows four models from twelve D8L8 and five D8L6 runs. Evaluation extends beyond the six or eight training loops without changing the weights.

Target-selection accuracy. Figure 3a uses the same 4,110 examples across the unsteered and steered conditions. The endpoint, one-hop, and two-hop labels are distinct, and examples are not filtered by prediction correctness. We classify each answer as the endpoint, its first successor, its second successor, or another node, and average the steered results across the two fits per target. The subsequent mechanism tests retain the same backbone and one-hop maps but use their own paired evaluation inputs.

Head selection and paired inputs. The mechanism tests reuse the frozen checkpoint and two one-hop maps specified above.

We select head H2 in the second layer on an earlier discovery set of 32 graphs. It has the highest mean eligible pattern-patching accuracy across the two map fits, with ties resolved by head index. H3 is the prespecified comparison head. The confirmation keeps this choice fixed and retains results for all four heads.

For confirmation, sampling seed 2026092404 draws 512 receiving graphs and 512 source graphs without replacement from a reserve of 17,519 unused permutation graphs. All 1,024 identities are excluded from backbone training, map training, earlier mechanism evaluations and their generated variants, and the preceding 1,024-graph evaluation of the main D8L6 backbone. The reserve was constructed by single swaps in earlier held-out graphs.

Each receiving graph is paired with an independently drawn source graph; the pair need not share most edges. Node slots align between inputs. We evaluate all ten current nodes, shifting the source current node by three modulo ten. Starting nodes are chosen by following each graph backward so that eight task steps end at the designated current node. Each run executes six loops, the all-token map, and a seventh frozen loop.

Scoring populations. These choices produce 5,120 candidate pairs before patching. The pattern-versus-output experiment requires three distinct answers: the original receiving answer, the source answer, and the receiving-graph answer under the source route. Both unpatched runs must correctly read their current and next nodes. This leaves 3,966 pairs for fit 1 and 3,963 for fit 2; within each fit, both patch types use exactly the same pairs.

For restoration, the alternative-current run must have correct current and next readouts and a distinct target. Recovery is scored among eligible clean transitions broken by query replacement, giving 4,413 and 4,417 cases. Steering-pattern exchange uses 4,044 examples with distinct current, one-hop, and two-hop labels, without conditioning on prediction correctness. The earlier target-selection figure uses a separate cohort of 4,110 examples. Each experiment retains its own denominator.

Uncertainty and implementation checks. For each map fit, we bootstrap 5,000 samples of the 512 receiving graphs together with their paired source graphs. All ten starting nodes of a graph remain together. The two fits measure variation between maps on one backbone, rather than between independently trained backbones.

Direct evaluation, reconstructed attention, an independent output hook, and same-run activation replacement agree on predictions when no effective intervention is made.

Fraction of steering gain recovered. For each steering-map fit, we compute $r = (a_{\text{patch}} - a_{\text{base}})/(a_J - a_{\text{base}})$, where a_{base} , a_J , and a_{patch} are the unsteered, fully steered, and steered-pattern-patching accuracies. We report the mean of the two fit-specific ratios, rather than a ratio computed from rounded aggregate accuracies.

A.4 SUPPORTING MECHANISM AND CONTROL RESULTS

A.4.1 RESTORING HEAD OUTPUTS RECOVERS ANSWERS

We test whether restoring selected head outputs repairs errors caused by query replacement.

We run the same graph with two different starting nodes. The original run provides the clean answer and head outputs. The alternative run provides replacement queries. We insert these queries into the original run while keeping its keys and values unchanged. This changes where attention reads without replacing the input’s value vectors. We then replace the selected heads’ disrupted outputs with their clean outputs and let the model continue. For an intervened head, let Q_{alt} denote the replacement queries and d_k the query/key dimension:

$$z_{\text{broken}} = \text{softmax}\left(\frac{Q_{\text{alt}}K_{\text{clean}}^\top}{\sqrt{d_k}}\right)V_{\text{clean}}, \quad z_{\text{restored}} \leftarrow z_{\text{clean}}, \quad (6)$$

Here softmax is taken over the visible key positions. If the selected outputs carry information needed for the answer, restoring them should repair the error caused by query replacement. We compare this repair with restoring a control set of heads. Recovery is measured only on examples that were answered correctly before query replacement and incorrectly afterward.

The restoration results show that restoring the selected outputs repairs answers in both models. In D8L6, it repairs 60.1% of disrupted answers, compared with 0.9% for the control. In Ouro, it repairs 62 of 63 answers, while the matched control repairs one. Restoring all affected outputs recovers every disrupted answer in both models. The selected head outputs therefore supply information that the later computation uses to produce the answer.

A.4.2 ONE-HOP AND TWO-HOP PATTERN EXCHANGE

We reuse the frozen seed-6 backbone and both pairs of target-specific maps. Sampling seed 2026092501 draws 512 fresh graphs from the training-excluded reserve, excluding previous evalu-

ation graphs and their generated variants. Evaluating all ten starting nodes yields 5,120 examples. We retain the 4,116 examples with distinct current, one-hop, and two-hop labels, without filtering by prediction correctness. Both directions and both map fits use this same population. This cohort is separate from the 4,110-example target-selection figure and the 4,044-example steering-pattern experiment.

Each source and receiver share the input and h_6 , but use different maps before the final frozen loop. We exchange post-softmax patterns at all token positions and all four heads of layer 1, layer 2, or both. Values are computed in the receiving run, and subsequent computation proceeds normally. When both layers are patched, each source pattern comes from the complete source run.

Same-run replacements preserve predictions, and replacing queries and keys together reproduces the corresponding pattern-patch predictions. Direct evaluation and the intervention implementation agree on unpatched predictions; checkpoint hashes remain unchanged. Confidence intervals use 5,000 paired bootstrap samples of graph identities, retaining all starting nodes and both map fits together. The two fits quantify map-fit variation on a fixed backbone.

Table 2: Donor-target answers after target-specific pattern exchange (percent). Arrows denote source to receiver. Confidence intervals describe the two-fit mean, with graphs as bootstrap units.

Direction	Patched layers	Fit 1	Fit 2	Mean	95% CI
$J_{\text{two}} \rightarrow J_{\text{one}}$	1	0.29	0.36	0.33	[0.17, 0.52]
$J_{\text{two}} \rightarrow J_{\text{one}}$	2	44.97	45.82	45.40	[44.43, 46.36]
$J_{\text{two}} \rightarrow J_{\text{one}}$	1 and 2	44.85	45.68	45.26	[44.31, 46.23]
$J_{\text{one}} \rightarrow J_{\text{two}}$	1	0.97	1.02	1.00	[0.72, 1.30]
$J_{\text{one}} \rightarrow J_{\text{two}}$	2	59.67	62.17	60.92	[59.84, 61.97]
$J_{\text{one}} \rightarrow J_{\text{two}}$	1 and 2	60.25	63.00	61.63	[60.53, 62.72]

Table 3: Second-layer pattern exchange across the five frozen D8L6 backbones on the same fresh cohort. Entries are donor-target percentages averaged over two map fits.

Backbone seed	$J_{\text{two}} \rightarrow J_{\text{one}}$	$J_{\text{one}} \rightarrow J_{\text{two}}$
6	45.4	60.9
3	15.3	0.0
4	50.0	52.2
5	21.3	0.4
7	25.2	2.1

The effect varies across backbones and directions. In seed 4, pattern patching also makes the receiver predict the source target in both directions. In seeds 3, 5, and 7, this effect is weaker and differs more between the two directions. These results support the routing account in the selected mechanism cases without making it a universal explanation of successful target steering.

A.4.3 COMPOSITION OF FROZEN TARGET-SPECIFIC MAPS

We evaluate all five D8L6 backbones and both existing pairs of one-hop and two-hop maps without further training. The first continuation applies a map to h_6 and executes loop index 6; the second applies a map to the resulting full token-state sequence and executes loop index 7. The input remains fixed, and no state reset or intermediate-answer input is used. Each map acts independently on all tokens. We also evaluate all combinations in which either map is omitted, giving nine two-loop conditions per fit.

This exploratory evaluation reuses the 512 training-excluded graphs from the target-pattern exchange experiment. All ten starting nodes are evaluated, yielding 5,120 examples. The common scoring population requires $u, f_G(u), \dots, f_G^4(u)$ to be pairwise distinct, leaving 3,200 examples for every condition and backbone. Inclusion never depends on prediction correctness. This stricter collision exclusion makes the denominator different from the single-continuation experiments. Targets are measured from the designated node u , not from the model’s first prediction. The no-second-map control is scored against the same cumulative target as its corresponding composition.

For every backbone and map fit, first-continuation predictions agree exactly with the earlier saved evaluations. Model parameters remain unchanged, and model, map, data, and code hashes are recorded. We bootstrap 5,000 paired samples of graph identities, preserving all starting nodes and both map fits within each sampled graph. Intervals quantify graph-sampling uncertainty for the two-fit mean on a fixed backbone.

Table 4: Composition across frozen backbones, averaging two map fits (percent). Columns name the first and second maps.

Backbone seed	One-one	One-two	Two-one	Two-two
6	93.64	65.28	74.50	68.12
4	83.78	40.98	55.02	28.42
3	22.14	10.53	46.22	97.44
5	18.39	6.83	28.64	83.58
7	20.00	9.45	28.14	91.47

Composition varies substantially across backbones and target sequences. Seeds 3, 5, and 7 reach high accuracy for two-two despite weaker one-one and mixed compositions. Reuse of a target-specific map therefore need not imply equally reliable switching between maps. These results test two successive continuations, not arbitrary-length closure of the controlled dynamics.

A.4.4 BACKBONE VARIATION AND CASE SELECTION

The five ten-node D8L6 backbones use training seeds 6, 3, 4, 5, and 7 under one backbone protocol. Each completed two map fits and the mechanism tests before case selection. We chose seed 6 for the clearest chain across restoration, semantic patching, and steering-pattern exchange and seed 4 for strong target control with much smaller gains from pattern patching. The screening results below document that choice.

Table 5: Earlier D8L6 screening results used to select cases, averaged over two steering-map fits (percent). Cross-graph eligibility counts are reported per fit out of 5,120 candidates. These data are separate from the fresh confirmation.

Seed	Full J	Pattern patching	Pattern: rerouted	Restore head	Eligible pairs
6	99.6	87.7	82.8	59.8	4986/4979
3	82.0	0.0	49.9	73.1	184/274
4	99.8	16.4	35.7	10.0	5040/5030
5	86.3	0.0	11.2	14.3	40/40
7	74.3	0.0	0.0	17.7	47/33

The selected models and their discovery-set heads remain fixed in the fresh confirmation on the 512 paired graphs defined in Appendix A.3. Seeds 6 and 4 are mechanism cases selected for contrasting results, rather than a random sample of seeds.

The seed-4 D8L6 backbone again shows that successful steering need not yield large gains from patching the tested patterns. Full steering reaches 99.6%, while steered patterns raise unsteered accuracy from 3.0% to 16.1%. Restoring its selected head repairs 9.6% of eligible broken answers, and cross-graph pattern patching selects the receiving-graph answer under the source route in 34.3% of eligible pairs. These outcomes confirm the contrast with seed 6 without identifying the remaining pathway responsible for seed 4’s successful steering.

Readout trajectories and pattern-transfer failures. The native readout trajectories suggest a distinction between regular two-hop progression and progression with variable increments. We apply the frozen final LayerNorm and readout head to the answer-token state after each layer, without steering. On 256 ten-node cycle graphs and all ten starting nodes, seeds 3, 5, and 7 predominantly read out depths 0, 2, 4, 6, 8 at successive loop ends, then retain depth 8. Their readouts usually remain at the previous endpoint after the first layer and change after the second. Seed 6 instead exhibits both one-hop and two-hop readout increments. Seed 4 has less regular intermediate readouts and

also permits target exchange in both directions, although its steered-to-unsteered pattern transfer is much weaker than seed 6’s. These intermediate readouts describe accessible predictions, not a complete account of the computation within each layer. Depth labels in this cycle-graph diagnostic are distinguishable only modulo ten.

The failures persist when controller capacity and head coverage are increased. We replace the rank-48 maps with dense affine maps $J(h) = hW + b$, using the same target losses, 8,000-update budget, and two fit seeds. The backbone remains frozen. On a separate 256-graph permutation cohort, all ten starting nodes yield 2,067 examples with distinct $u, f_G(u), f_G^2(u)$, without filtering by correctness. Full dense one-hop steering exceeds 99.8% for seeds 3, 5, and 7. Yet one-hop pattern transfer from $F(J_{\text{one}}(h_6))$ to $F(h_6)$ scores zero for every nonempty subset of the eight heads in the final loop. This holds when replacing either the answer-query rows or all query rows, in both map fits. Values are not directly replaced, and no residual attenuation is used.

Table 6: Dense one-hop control and pattern-transfer failure on the same 2,067 examples, averaged over two fits (percent). Pattern transfer reports the maximum over all subsets of the eight heads and both query-position scopes.

Seed	Full J_{one}	One-hop pattern transfer
3	99.90	0.00
5	99.90	0.00
7	99.88	0.00

These results motivate a mechanism hypothesis: backbones with regular two-hop progression may implement a coupled update that is difficult to redirect into a one-hop transition through routing alone. Backbones with variable readout increments may offer more ways to select different amounts of progress through routing. This interpretation is consistent with the stronger target exchange in seeds 6 and 4 and the strong two-two composition in seeds 3, 5, and 7. It does not establish that the latter lack one-hop computation: full J_{one} succeeds. The distinction concerns whether one-hop behavior can be elicited by changing patterns alone or requires coordinated changes to other state components.

B OURO LETTER-WALK

This appendix describes the language-based graph task and attention interventions used in Section 5.

B.1 TASK DEFINITION AND INPUT FORMAT

Each example defines a cycle over ten people using ten sentences of the form “Alice passes the letter to Bob.” The input also names the starting person and the number of steps to follow. Names are Alice, Bob, Carol, David, Emma, Frank, Grace, Henry, Iris, and Jack. Rules appear in shuffled order, so their textual order does not give the path. Ouro’s tokenizer and chat template encode the instructions, rule sentences, and question as a user message. The target is an assistant message containing only the final person’s name.

For example, take the cycle Alice $\rightarrow$ Bob $\rightarrow$ Carol $\rightarrow$ David $\rightarrow$ Emma $\rightarrow$ Frank $\rightarrow$ Grace $\rightarrow$ Henry $\rightarrow$ Iris $\rightarrow$ Jack $\rightarrow$ Alice. The prompt includes all ten rules. Starting with Alice, the two-step answer is Carol and the eight-step answer is Iris. For illustration, the question can be phrased as:

The letter starts with Alice. After exactly 02 steps, who has it? Answer directly with only the person’s name.

This example paraphrases the question wording; 02 preserves the template’s two-digit encoding of the requested step count. The rules and question are ordinary text, not one special token per person or per edge. Different prompt templates express the same task.

B.2 MODEL AND STEERING CONFIGURATION

Supervision and steering. Backbone fine-tuning uses requests for 1–4 steps, four recurrent loops, and answer-token cross-entropy at loop 4 only. Prompt tokens are masked from the task loss. The

target includes the assistant answer and the chat-template ending, without an intermediate reasoning trace. We then freeze the backbone and fit one dense linear layer on requests for 1–8 steps. The same map acts on every token before loops 2–4, and answer supervision remains at loop 4. Both stages combine task-answer cross-entropy and general-text cross-entropy with weights 0.8 and 0.2. Thus, an eight-step request is beyond backbone task training but within map training. The mechanism evaluations use eight-step questions; their attention interventions change internal activations, not the requested task or its answer rule.

Checkpoint and intervention sites. All Ouro mechanism tests use the letter-walk Ouro-2.6B checkpoint saved after 200 backbone updates and its dense linear layer saved after 500 map updates. The frozen map is $J(h) = hW + b$ with unrestricted W , applied to all token states before loops 2–4. Head outputs are patched before their output projection during prompt processing; generation follows the receiving run’s normal steering schedule.

B.3 EVALUATION AND ATTENTION INTERVENTIONS

Head selection. Using zero-based layer and head indices, the selected set is L41.H2/H10/H12/H15, L43.H1/H6/H11/H13/H15, and L47.H0/H2/H3/H4/H7/H13/H15, all at loop 4. Selection begins with eight discovery pairs and uses a separate 64-pair pattern-patching confirmation set. The subsequent semantic and restoration tests keep the sixteen-head set fixed.

The semantic and restoration experiments use sixteen comparison heads matched by layer to the selected set.

Restoring outputs after query replacement. Restoration uses 64 same-graph query pairs generated with seed 2026092701. This set is separate from localization, but was previously used for an L34.H5 restoration test. At loop 4, we replace all queries in layers 41, 43, and 47 while retaining the receiving keys and values. We then restore clean outputs at the sixteen selected heads, sixteen layer-matched comparison heads, or all 48 affected heads. Recovery conditions on clean answers broken by query replacement; restoration outcomes do not determine inclusion.

Pairs that separate routing from retrieved content. The semantic experiment draws 64 new pairs using seed 2026092901, excluding all 544 graph identities from earlier exploration, semantic and restoration panels, and localization confirmation. Each graph is a ten-person cycle. From the source starting person, the recipients at the first seven steps agree across graphs and the eighth recipient differs. The receiving query starts elsewhere. Names occupy aligned token slots.

The construction makes three answers distinct: the original receiving answer, the source answer, and the receiving-graph answer reached from the source start. A reference run with the receiving graph and source start evaluates the third answer. Pair inclusion is independent of model accuracy. This specifies the semantic predictions of pattern and head-output patching without assigning one internal loop to each task step.

Numerical checks and statistical reporting. Same-run output replacement and restoration of every affected output both yield zero maximum logit error. Pattern replacement is checked for prediction invariance, allowing floating-point differences from explicit attention reconstruction.

Ouro figure intervals are Wilson intervals. When full names are generated, we compare them with first-token predictions and count truncated generations as incorrect. The historical backbone has no verified graph-exclusion list establishing separation of these graph identities from its training set.

B.4 ADDITIONAL RESULTS

Head-output restoration. The table reports both accuracy over all query pairs and recovery among clean answers broken by query replacement.

Table 7: Restoring Ouro head outputs at the selected positions. The last column conditions on cases broken by query replacement.

Condition	Correct /64	Recovered / broken
Clean	63	—
Replaced queries	0	0/63
Selected 16 heads	62	62/63
Matched 16 neighbors	1	1/63
All 48 affected heads	63	63/63

Pattern-versus-output patching. The table separates the original, corrupted, and rerouted answers for the same paired inputs.

Table 8: Ouro pattern-versus-output patching, all 64 pairs retained. Counts use first-answer-token predictions. “Rerouted” means the answer obtained by following the source route on the receiving graph.

Condition	Original	Source	Rerouted	Other
Receiving run	61	0	1	2
Source run	0	63	0	1
Receiving graph, source start	0	2	61	1
16-head pattern	10	0	46	8
16-head output	0	61	0	3
Matched-neighbor pattern	61	0	1	2
Matched-neighbor output	61	0	1	2

For the selected sixteen heads, complete-name accuracy agrees with first-token scoring: pattern patching gives 46/64 matches to the rerouted answer and zero to the source answer; output patching gives zero and 61/64, respectively. The two interventions therefore distinguish the same answers under both scoring rules.

Steering-pattern transfer. On the 64 confirmation pairs, native and fully steered accuracy are 0/64 and 62/64. Patching the selected sixteen heads from the steered run into the unsteered run gives 61/64; the reverse patch gives 0/64. These counts score the first answer token. Full names were not recorded for this sixteen-head steering-pattern test, but the later semantic and restoration tests retain complete-name checks.

C PARITY

This appendix defines the parity task in Section 6.1, describes the readout-timing analysis, and reports results for all three backbones.

C.1 TASK DEFINITION AND INPUT FORMAT

A parity input is a sequence of binary tokens followed by a separator and, when needed, padding. The token IDs are 0 and 1 for bits, 2 for the separator, and 3 for padding. For example, the token sequence 1 0 1 1 2 has parity target 1 at the separator because it contains three ones. The sequence 1 1 0 0 2 has target 0. With batch padding, the first example can instead be 1 0 1 1 2 3 3; the supervised output at and after the separator is then 1 3 3. Input-bit positions are excluded from the answer loss. The semantic answer is the parity bit; the implementation also supervises the padding suffix and scores the complete answer region.

C.2 MODEL AND STEERING CONFIGURATION

For an input of length n , the loss reads the answer after n loops. Backbone training uses lengths 1–20; map training freezes that backbone and uses lengths 20–40. Both use cross-entropy on the answer

region. The same rank-48 diagonal-plus-low-rank linear layer is applied before loops 2 through n . No intermediate loop is assigned a partial-parity target. The long-length experiment keeps the learned map fixed and increases both the input length and the requested loop count.

Our parity task and length-dependent loop schedule follow Fan et al. (2025). We use causal attention without positional encodings. Embeddings enter only at the first loop, rather than being reinjected at each loop. Three independently trained backbones are each paired with one fixed steering map.

C.3 EVALUATION AND READOUT-TIMING ANALYSIS

Dense evaluation from length 500 to 1000. The extended curve uses seed 2’s input-once backbone at update 99,000 and its map fitted on uniformly sampled lengths 20–40. We evaluate 101 lengths, 500, 505, $\dots$, 1000, with 128 binary inputs per length. Native and steered conditions share inputs, generated with seed 2026092401 + 1009 n , and retain all 12,928 sequences. Embeddings enter only at loop 1; J acts before loops 2 through n .

Through length 500, each length uses 64 examples and the main curve uses a 10-length moving mean. From length 500 onward, every fifth length uses 128 paired examples; the dark curves average adjacent measured points, and faint points retain individual measurements. Gray shading in Figure 5a marks map-training lengths 20–40.

Heatmaps and readout offset. The parity heatmaps use 256 examples for each input length and loop count up to 60. Insets display lengths and loops 54–60 without interpolation; dots mark the unique best loop among $t = n - 1, n, n + 1$ within the measured grid. The dotted horizontal line marks the longest map-training length, $n = 40$.

For each seed, the heatmaps, long-range curves, and phase measurements use the same saved map and intervention schedule. We estimate band positions using the period-four Fourier component along the offset between loop number and input length. Unwrapping the phase over input length gives a continuous band-position estimate, to which we fit a line over lengths 41–490.

Confidence intervals use 5,000 moving-block bootstrap samples of fit residuals, with block length 20. The estimate summarizes the visible accuracy bands; it does not identify an oscillator in the hidden state. The additional results below report the fitted drift, residual error, and requested-loop accuracy for all three seeds.

C.4 READOUT TIMING ACROSS BACKBONES

Long-range accuracy and readout timing.

Table 9: Readout-schedule drift and longer-range accuracy with each fixed steering map. Drift is measured in loops per 100 tokens; MAE is the mean absolute deviation from the fitted phase line. Accuracy averages $t = n$ over lengths 101–500.

Seed	Drift: native $\rightarrow J$	MAE: native $\rightarrow J$	Accuracy: native $\rightarrow J$ (%)
0	+0.573 $\rightarrow$ -0.327	0.065 $\rightarrow$ 0.069	35.27 $\rightarrow$ 55.71
1	+1.312 $\rightarrow$ +1.731	0.174 $\rightarrow$ 0.743	44.03 $\rightarrow$ 45.24
2	-0.961 $\rightarrow$ -0.183	0.066 $\rightarrow$ 0.052	45.05 $\rightarrow$ 73.73

Seed 1 illustrates a difference between improving requested-loop accuracy and aligning the longer-range schedule. Its mean accuracy improves slightly, but its drift and residual error both increase with J , and a single linear trend poorly describes its later readouts.

D GRAPH CONTINUATION

This appendix describes the continuation experiment in Section 6.1. These eight-node D8L8 models are separate from the ten-node D8L6 models used for target selection and attention interventions.

D.1 TASK DEFINITION AND INPUT FORMAT

This experiment uses the graph-token format in Appendix A.1, with eight nodes and 29 input tokens. The query continues to specify depth 8 even when execution extends beyond eight loops. At loop t , the continuation target is the node reached after t graph edges from the original start. The input is not rewritten with a new requested depth at each loop.

For example, on the cycle $0 \rightarrow 1 \rightarrow \dots \rightarrow 7 \rightarrow 0$ with start 0, the loop-8 target is 0, the loop-9 target is 1, and the loop-10 target is 2.

D.2 MODEL AND STEERING CONFIGURATION

The continuation study uses twelve eight-node D8L8 backbones with seeds 100–111, separately from the ten-node D8L6 mechanism models. Each has two shared blocks, hidden width 256, and four heads, and reaches perfect endpoint accuracy at its supervised eighth loop. Two rank-48 diagonal-plus-low-rank maps are fitted for each frozen backbone using successor cross-entropy.

The backbone is trained to answer the depth request at loop 8. Map fitting instead supervises successive nodes over loops 1–16, using states generated by the steered run itself. The controller is applied before every loop, including loop 1 and evaluation loops beyond 16. This differs from the single extra loop used for D8L6 target selection.

D.3 EVALUATION PROTOCOL

The main evaluation draws 8,192 unique graphs uniformly without replacement from all 40,320 eight-node permutations, using seed 2026092402. It contains 993 single cycles as well as graphs with shorter cycles and fixed points. All eight starting nodes and all predictions are retained. Evaluation reuses the twelve original checkpoints and their two saved maps, with steering starting before loop 1.

Counts are stored by graph, loop, backbone, and map. We average over starting nodes and graphs, then over maps and backbones; the plotted 95% interval bootstraps the twelve backbone means 10,000 times. Gray shading in Figure 5b marks loops 9–16, the displayed portion of the map-fitting range; the map remains active beyond this range. This broad sample has not been checked against historical training exclusions. The additional results below also report a separate training-excluded population.

D.4 VARIATION ACROSS BACKBONES AND EVALUATION SETS

Across the twelve backbones, steered accuracy ranges from 75.01% to 100.00% over loops 9–16 and from 34.73% to 54.56% over loops 17–32. Each value averages the two fixed maps. Every backbone has lower mean accuracy beyond the map-fitting range.

A separate training-excluded evaluation. A cohort of 512 graphs is excluded from backbone training, map training, and protocol selection. Its 78 single-cycle graphs give 97.42% steered accuracy over loops 9–16 and 35.11% over loops 17–32. Native execution averages 12.5% in each range. All twelve backbones decline beyond the fitted range.

This subset is selected by graph structure, not successful predictions, and is reported separately from the main broad-distribution curve. Scores are computed at each loop. On an eight-node cycle, retaining one endpoint gives a correct answer once every eight loops; the resulting 12.5% does not require sustained traversal.

E OURO ANTONYM CANCELLATION

This appendix defines the task and the two supervision strategies compared in Section 6.2.

E.1 TASK DEFINITION AND INPUT FORMAT

The input contains a word sequence, a deletion rule, and a requested deletion count. At each step, the rule removes the leftmost adjacent antonym pair and joins the remaining words in their original order.

The vocabulary consists of twelve pairs: hot/cold, near/far, open/closed, heavy/light, young/old, rich/poor, happy/sad, clean/dirty, full/empty, wet/dry, fast/slow, and alive/dead. Actual examples contain all 24 words, with pair orientations and nesting varied. Ouro encodes the full text using its tokenizer and chat template. The answer contains the two words removed at the requested step, in their left-to-right order.

A short illustrative sequence is `hot near far cold open closed`. The first deletion removes `near far`, leaving `hot cold open closed`. The second removes `hot cold`, and the third removes `open closed`. Thus, the answer to deletion 2 is `hot cold`, not the remaining sequence. This shortened example illustrates the rule; training and evaluation use 24-word inputs. In the final-only format, the question is “Which two words are deleted on deletion number 2? Answer with only those two words in their original left-to-right order.” The stepwise format instead states the requested number of deletions and asks for the pair removed in the current deletion update.

E.2 MODEL AND STEERING CONFIGURATION

Answer-token supervision. The user prompt is masked from the task loss. Cross-entropy supervises the assistant’s two-word answer and chat-template ending, without a generated deletion trace.

Backbone supervision. The two Ouro backbones are fine-tuned separately for 500 updates and always run for four loops. Final-only training samples requests for deletions 1–4 and supervises the requested pair at loop 4. Stepwise training supplies a prompt requesting four deletions and supervises the pair removed at deletion t after loop t , for $t = 1, \dots, 4$. The comparison therefore contrasts learning to answer a requested depth at the final loop with learning a prescribed progression of intermediate answers.

A shared map for each frozen backbone. Each backbone receives its own map $J(h) = h(I + AB) + b$, with a rank-128 residual update and 526,336 trainable parameters. It is shared across all four loops and acts on every token after each loop’s RMSNorm, including after loop 4 before the frozen language-model head. Zero initialization of B and b makes the initial map the identity.

Both fits use 500 AdamW updates at learning rate 10^{-4} , with 50 warmup updates. An update contains sixteen examples, two for each requested deletion depth 1–8. The loss weights answer cross-entropy by 0.8 and general-text cross-entropy by 0.2. During both map fitting and evaluation, the prompt specifies the requested deletion count. For both backbones, map training supervises the requested pair at the fixed loop-4 exit. Adaptive halting is disabled so all requests use four loops.

E.3 EVALUATION PROTOCOL

Native readout heatmaps. Figure 7a,b uses the trained, unsteered backbones at update 500. In the stepwise panel, the prompt requests four deletions. For each of 64 sequences, we compare the parsed pair read at each loop against each of the four true deletion pairs, producing the depth-by-loop matrix. In the final-only panel, the prompt requests the deletion depth shown on the vertical axis; each cell contains 64 examples from the archived loop-1/2 and loop-3/4 readout evaluations. They score the parsed deletion pair, not exact full-text generation or EOS emission.

Paired evaluation at deeper requested deletions. The main evaluation uses prompts that specify a deletion count and ask for the pair removed at that step (template 0 in the implementation). At each depth, the same 64 held-out sequence structures and target pairs are evaluated in all four conditions: each backbone with and without its map. Success requires matching the requested deletion pair. Figure 7c reports the evaluation at map update 500.

Each strategy is represented by one backbone and one map. Depths 5–8 extend beyond backbone task training but are included in map fitting. The low stepwise steering accuracy establishes failure of the tested map and fitting procedure; it does not by itself prove that no alternative state intervention could elicit deeper computation.